Inference-Time Scaffolding for Small-Model Math Reasoning: It's the Termination, Not the Tools

Anonymous authors
Paper under double-blind review

Abstract

We study what actually drives the gains of multi-tool inference-time scaffolding on small math LMs. On GSM8K, wrapping a 2B-parameter Qwen3.5 student (already trained with on-policy distillation and a GRPO variant to 78.47%) in a two-tool agent harness—a Python interpreter and an inversion-style validator—lifts accuracy to 86.05% (+7.58pp); the same harness on the 4B teacher lifts it from 83.62% to 93.86% (+10.24pp). With n=1319 and binomial standard error $\approx$ 0.95pp, we ablate each component and find a counterintuitive pattern: removing the Python tool changes accuracy by +0.53pp ($< 1\sigma$), removing the validator changes it by +0.15pp ($< 1\sigma$), removing both tools simultaneously changes it by +0.15pp ($< 1\sigma$, 86.20%), but removing a single ~80-token *post-validation nudge* that forces termination drops accuracy by 4.25pp ($> 2\sigma$). On the SVAMP out-of-distribution set, removing both tools *improves* accuracy by 3.5pp (87.50% $\rightarrow$ 91.00%). On MATH-500, also OOD and substantially harder, removing both tools improves accuracy by 3.4pp (63.80% $\rightarrow$ 67.20%). On Llama-3.2-3B-Instruct, removing both tools improves accuracy by 15.6pp (41.70% $\rightarrow$ 57.32%). Across four settings the tools never causally improve accuracy and often hurt it; the post-PASS nudge is the only intervention with a consistently positive effect, and only on the configurations where the model otherwise enters a "double-check spiral" after `[VALIDATION PASS]`. We further compute a binary-Oracle-EFC proxy on 1950 pooled traces and find a monotonic Q1$\rightarrow$Q4 accuracy increase of +6.98pp by EFC quartile, while raw token counts negatively correlate with success (r_{pb}=$-$0.21). These results directly validate, at the mechanism level, the Effective Feedback Compute scaling law of Zhang et al. (2026): tool-augmented scaffolds add raw compute without adding effective feedback in this regime, and the field's vocabulary of "tool-augmented reasoning" attributes the lift to the wrong component.

1 Introduction

Recent work on small-LM math reasoning has converged on a recipe of distillation/RL followed by inference-time tool use: Program-of-Thought (Chen et al., 2022), ToRA (Gou et al., 2023), ReST-style self-improvement, and various "agent" wrappers around chat-format LMs all combine a Python interpreter with some form of verification or self-reflection. Reported gains over chain-of-thought (CoT) baselines are large (+5 to +15pp on GSM8K), and the dominant narrative attributes these gains to the *tools*—to Python's exactness, or to the verifier's catching of arithmetic errors.

This paper questions that attribution. We build a deliberately simple two-tool harness—`python` + an inversion-checking `validate` tool—on top of an already-distilled 2B model, observe the expected +7.58pp lift, and then ask: *which component is doing the work?*

The answer is uncomfortable for the standard story. **We cannot reject the null that either tool, on its own, contributes nothing.** Removing `python` (forcing the model to use only `validate`) yields 86.58%, slightly *above* baseline ($\Delta = +0.53$pp, well within the binomial standard error of $\sim$0.95pp). Removing `validate` (leaving only `python`) yields 86.20% ($\Delta = +0.15$pp). But removing a single post-validation

nudge—an 80-token user message that fires once after `[VALIDATION PASS]` to force `\boxed{}` emission—drops accuracy by 4.25pp, an effect of more than two standard errors and qualitatively robust across re-runs.

The mechanism is mundane: without the nudge, the model after receiving `[VALIDATION PASS]` keeps invoking tools to "double-check," exhausting `max_turns` and never reaching a clean boxed answer (Appendix B). The validate tool's `verification_code` parameter is itself a Python snippet that the runtime executes, so "remove `python`" does not in fact remove Python execution from the loop; the model just routes its computation through `validate`'s argument. The two tools are most naturally read as redundant Python-execution channels with one of them additionally exposing a PASS/FAIL judgment that the nudge can hook onto.

Our contributions:

1. A clean +7.58 / +10.24pp Harness Agent result on GSM8K with a 2B-OPD-GRPO student and the 4B teacher (§3).
2. A four-way ablation table (§4) showing that removing either tool individually changes accuracy by less than one standard error, while removing the post-PASS nudge drops it by 4.25pp ($> 2\sigma$). We propose a tentative decomposition—$\sim$+4.3pp from forced termination, $\sim$+3.3pp from the multi-turn framework, $\sim$0pp from each individual tool—under an additivity assumption we cannot fully verify (§4.1).
3. A reframing of "agent for math": the productive levers, in this setup, are (a) some Python execution channel (which either tool supplies), (b) multi-turn retry budget, and (c) a forced termination mechanism. Tool *identity*, holding the Python channel constant, is not the lever (§5).

The paper is deliberately narrow in scope: one dataset (GSM8K, 1319 test samples), one model family (Qwen3.5), and one ablation axis (component removal, not variation). Section 8 discusses what this scope can and cannot support, including the one missing ablation cell (no-tool, nudge-only) that would distinguish "the Python channel is necessary" from "the nudge alone suffices."

2 Background and Setup

2.1 The training pipeline (backbone, not the contribution)

The student backbone is **Qwen3.5-2B**, trained in two stages:

1. **On-Policy Distillation (OPD)** (Agarwal et al., 2023) from a Qwen3.5-4B teacher. Student samples a trajectory, teacher provides per-token reference logits, KL pushes the student toward the teacher distribution at each step. One full epoch on the GSM8K training split.
2. **GRPO variant** with two modifications: Dr.GRPO (Liu et al., 2025) (removes per-sequence length and within-group reward standardization), and DAPO Clip-Higher (Yu et al., 2025) (asymmetric PPO clip with $\epsilon_{lo} = 0.2$, $\epsilon_{hi} = 0.28$ to allow upside ratio updates). Trained 400 steps on a Boolean (boxed-correct) reward; we report the peak-checkpoint (ckpt-400) score.

Baseline pretrained, OPD-only, and OPD+GRPO numbers are summarized in Table 1. The 0.8B model is trained identically and reported for scale context but is not the focus.

The 2B+OPD+GRPO student closes 62% of the 2B-to-4B gap on raw CoT, but stalls at 78.47%—within $\sim$5pp of the 4B teacher. Diagnostic measurements of the token-level Top-K overlap between student and teacher distributions ($m_{\text{overlap}} = 0.71$ on held-out completions) suggest the student is at or near the capacity ceiling reachable from this teacher via on-policy distillation.

2.2 Harness Agent

The harness exposes two tools to the model through the standard OpenAI tool-calling interface:

Table 1: GSM8K test accuracy along the training pipeline (no scaffolding).

Stage	0.8B	2B	4B
Pretrained base	52.92	69.98	83.62
+ OPD (1 epoch)	62.55	75.97	— (teacher)
+ OPD + GRPO variant	64.52	**78.47**	—

- `python(code: str) -> str`: Run the code in a subprocess (5 s timeout, only `math` builtin) and return stdout. Used to compute a candidate answer via the *forward* formulation.
- `validate(answer: str, verification_code: str) -> str`: Run `verification_code` (also Python, same sandbox), parse its last printed number, and compare to `answer`. Return `[VALIDATION PASS]` iff they match, else `[VALIDATION FAIL]`. The system prompt is explicit that `verification_code` must verify by *inversion* (plug `answer` back into the problem's stated constraints and check), not by re-running the same forward formula.

Two control-flow patches sit on top of the tool layer:

- **Post-PASS nudge.** When a `validate` call returns `[VALIDATION PASS]`, the loop appends a single short user message: *"STOP calling tools. Output `\boxed{<number>}` on its own line."* This fires exactly once per trajectory.
- **Coda.** If `max_turns` (=10) is exhausted without a boxed emission, a final user message forces a single boxed-only completion.

The system prompt walks through one worked example (Josh's-house-profit, a classic "verify the forward formula instead of the constraint" trap) and demands inversion-style verification.

2.3 Evaluation protocol

All accuracy numbers below are on the full GSM8K test split (1319 problems), evaluated against gold via numeric tolerance ($|p - g| < 10^{-4}$ after stripping commas). The agent runs through vLLM 0.21 with the merged Qwen3.5-VL-base + trained-LM weights (the text-only checkpoint cannot be served by stock vLLM due to a hybrid-attention limitation). Sampling: temperature 0.7, top-p 1.0, max-tokens 2048, max-turns 10. Concurrency 128. Each ablation in §4 is a single full-1319 run; sampling noise is bounded by $\sqrt{p(1-p)/n} \approx 1.0$ pp in this regime.

3 Main Results

Table 2: Effect of the Harness Agent across student and teacher.

Model	base CoT	+Harness	Δ
2B + OPD + GRPO	78.47	**86.05**	+7.58
4B teacher	83.62	**93.86**	+10.24

Two observations:

1. A 2B model with scaffolding outperforms the 4B teacher's raw CoT (86.05 > 83.62). The scaffold does not depend on the teacher's strength.
2. The lift is larger for the stronger model (+10.24 on 4B vs +7.58 on 2B). The standard interpretation would be "the smarter model uses tools better"; we revisit this in §5.

Auxiliary metrics for 2B+Harness on the 1319 test set:

- Validation rate (a `validate` call returned PASS): 92.27%
- Acc conditional on validated: 86.69%
- Acc conditional on unvalidated: 78.43%
- Trajectories using `python` at least once: 68.4%
- Trajectories using `validate` at least once: 96.0%

The 96.0% `validate`-usage rate vs 68.4% `python`-usage is the first hint that "the validator is doing the heavy lifting"—but §4 shows this hint is misleading.

4 Ablation: What Drives the Lift?

We construct three orthogonal ablations on the 2B+OPD+GRPO+Harness setup, each removing exactly one component while leaving the other two intact:

- **−python:** Only `validate` is exposed. The model's only Python channel is the `verification_code` parameter.
- **−validate:** Only `python` is exposed. No inversion gate; the model emits `\boxed{}` whenever it chooses.
- **−nudge:** Both tools present; the post-PASS nudge is suppressed. After `[VALIDATION PASS]`, the model is free to continue calling tools.

All other parameters held constant. Table 3 is the central result of the paper.

Table 3: Component ablation on 2B+OPD+GRPO. All numbers on the full GSM8K test split ($n = 1319$, binomial SE ≈ 0.95pp).

Configuration	Acc	Δ vs Harness	Val rate	py used	val used
Base CoT (no scaffolding)	78.47	−7.58	—	—	—
Harness (full)	86.05	—	92.3	68.4	96.0
− python tool only	**86.58**	**+0.53**	94.0	0.0	95.3
− validate tool only	**86.20**	**+0.15**	0.0	10.9	0.0
− both tools (no-tools)	**86.20**	**+0.15**	0.0	0.0	0.0
− post-PASS nudge	**81.80**	**−4.25**	89.5	63.2	91.0

The binomial standard error at $p \approx 0.86$, $n = 1319$ is $\sqrt{p(1-p)/n} \approx 0.95$pp. We use 1σ as our criterion for "within noise." Reading row by row:

- **Removing `python` does not hurt.** $\Delta = +0.53$pp is below 1σ. With only `validate`, the model still has Python execution (via `verification_code`); the only thing lost is a redundant compute pathway.
- **Removing `validate` does not hurt either.** $\Delta = +0.15$pp, well below 1σ. With only `python`, the model emits boxed without a PASS gate, and never validates (val rate = 0). Strikingly, only 10.9% of trajectories even *call* `python`; the model defaults to its CoT pathway and the agent format just provides longer context to think in.

- **Removing both tools does not hurt.** The same $\Delta = +0.15$pp as −validate. This was the cell we explicitly flagged as missing in our previous draft; we filled it. Once both tools are removed, the model receives no tool feedback at all—and yet matches the full Harness within sampling noise. The implication: in this saturated regime, the *Python channel itself is not required*. The agent format around the model (extended prompt, multi-turn retry budget, boxed-emission convergence) suffices.
- **Removing the nudge hurts substantially—4.25pp ($> 2\sigma$).** All other components intact, `validate` still usable, but the post-PASS user message is suppressed. The trajectories now show the failure mode the nudge was designed to prevent: after receiving `[VALIDATION PASS]`, the model keeps invoking `python` and `validate` to "make sure," exhausts the 10-turn budget, and is then forced into a degraded coda completion (worked trace in Appendix B). Note that −nudge is *strictly worse* than −no-tools (81.80 vs 86.20)—adding tools without a termination mechanism is a net negative.

4.1 Decomposition of the +7.58pp lift

With the −no-tools cell filled, the decomposition becomes cleaner. Anchoring on the −no-tools configuration as the "agent framework, no tools, no PASS events possible" baseline:

$$\begin{aligned} \Delta_{\text{framework alone}} &= 86.20 - 78.47 = +7.73 \text{ pp} \\ \Delta_{\text{tools (any)}} &\approx 0 \\ \Delta_{\text{nudge}} &= 86.05 - 81.80 = +4.25 \text{ pp once tools are added} \end{aligned}$$

The full +7.58pp lift is recovered without any tool execution at all. The agent format—extended prompt, multi-turn retry budget, retry-on-no-boxed nudges, worked-example pedagogy in the system prompt—accounts for the entire signed gain. Adding tools *without* a termination mechanism (the −nudge cell) *degrades* accuracy by 4.40pp relative to −no-tools, because the model enters PASS spirals that waste the turn budget. Adding tools *with* the post-PASS nudge restores accuracy to the −no-tools baseline—the nudge's causal role is not to add capability but to neutralize the harm tools introduce when not properly terminated.

4.2 Why does removing either tool individually not hurt?

The structural explanation is that `validate(answer, verification_code)` executes `verification_code` as Python under the hood. So −python does not actually remove Python execution from the trajectory; the model just routes its computation through `validate`'s argument. Conversely, in −validate, the model still has `python` (though it uses it only 10.9% of the time, relying mostly on direct CoT). With the no-tools cell also at 86.20%, however, the stronger claim survives: the Python channel itself is not even necessary in this regime—the multi-turn agent format alone suffices.

4.3 Cross-family and out-of-distribution generalization

We replicate the central comparison "full Harness vs −nudge vs −no-tools" on two additional axes (Table 4). *Same-family OOD*: SVAMP (1000 elementary word problems, 1∼2-step arithmetic, drawn from a different surface distribution than GSM8K). *Cross-family ID*: GSM8K test on Llama-3.2-3B-Instruct, with the same harness but without OPD/GRPO training (we use the off-the-shelf instruct checkpoint).

Table 4: Generalization. −nudge and −no-tools relative to full Harness, across four settings.

Setting	Model	full	−nudge	−no-tools
GSM8K (ID, n=1319)	Qwen3.5-2B+OPD+GRPO	86.05	81.80	86.20
SVAMP (OOD, n=1000)	Qwen3.5-2B+OPD+GRPO	87.50	87.40	**91.00**
MATH-500 (OOD, n=500)	Qwen3.5-2B+OPD+GRPO	63.80	61.60	**67.20**
GSM8K (n=1319)	Llama-3.2-3B-Instruct	41.70	41.39	**57.32**

Two findings replicate cleanly:

- **No-tools is never worse than full Harness, and is often substantially better.** +3.50pp on SVAMP, +3.40pp on MATH-500, +15.62pp on Llama. The GSM8K-on-Qwen result (the original setting) is the *least* striking case; the pattern is more extreme on harder benchmarks and on cross-family transfer.
- **The −nudge effect is specific to the PASS-spiral failure mode.** On SVAMP, where the model rarely spirals, −nudge ≈ full (within 0.10pp). On MATH-500, the gap is small (−2.20pp) but still smaller than the no-tools improvement. On Llama, where the model's tool calls are weaker but the model still calls validate at 89.2% rate, −nudge ≈ full (−0.31pp); the spiral mode is not what dominates here—tool exposure itself is poison.

Taken together, the four settings rule out the explanation that "tools help, just less than people thought." Across all four, the tools' causal contribution to accuracy is either zero (Qwen GSM8K) or negative (Qwen SVAMP, Qwen MATH-500, Llama GSM8K).

Capability-floor note: AIME 2024/2025. We also ran the same three configurations on AIME 2024 and AIME 2025 (combined n=60 problems, 30 each), where Qwen3.5-2B is far below ceiling: pooled accuracy is 21.7% for full Harness, 20.0% for −nudge, and 18.3% for −no-tools. The pooled differences are well below the binomial standard error at n=30 per split ($\sigma \approx 7.3$pp), and on the individual splits the direction of the nudge effect flips between 2024 (where full > −nudge by 10pp) and 2025 (where −nudge > full by 7pp). We interpret this as a capability-floor regime where the model's success rate is dominated by the small fraction of problems it can solve at all, and the choice of scaffolding mostly reshuffles which ∼5–8 of 30 problems it happens to get; the ablation framework we use does not deliver a useful signal in this regime. We do not include AIME in the main claim of the paper.

For size-controlled context, Table 5 places our 2B+OPD+GRPO+Harness AIME 2024 number (26.67%, 8/30) against reported AIME 2024 scores for other models. The single-seed n=30 result places our 2B system above GPT-4o and below the substantially larger and math-specialized Qwen2.5-Math-72B (greedy). We emphasize the $\sigma \approx 7$pp uncertainty: the difference between rows within a ∼10pp window is not statistically distinguishable from our single-seed data, and the comparison is included only to anchor the absolute scale of our numbers, not to argue ranking.

Table 5: AIME 2024 size-controlled context. Our 2B number is single-seed, n=30, σ≈7pp. External numbers are as reported by the cited sources; methodology (greedy / pass@1 / cons@k) varies across sources.

Model	AIME 2024	Source / note
GPT-4.5	36.7%	Artificial Analysis leaderboard
GPT-4.1 nano	29.4%	Artificial Analysis leaderboard
Qwen2.5-Math-72B-Instruct (greedy CoT)	30.0%	Qwen2.5-Math technical report
Qwen3.5-2B+OPD+GRPO+Harness (ours)	**26.67%**	this work, n=30, single seed
Qwen2.5-7B-Instruct (non-math)	16.67%	Qwen2.5-Math technical report
GPT-4o (pass@1)	13.1%	Artificial Analysis leaderboard
Claude 3 Opus / GPT-4 Turbo / Gemini 1.5 Pro	3–7%	Qwen2.5-Math technical report

The takeaway from this size-controlled view is not a ranking claim but a scale anchor: a 2B general-purpose model with our OPD+GRPO pipeline plus the Harness Agent sits roughly in the same accuracy range on AIME 2024 as the 72B math-specialized Qwen variant (greedy decoding) and substantially above GPT-4o, while remaining below frontier math-specialized models such as GPT-4.5 and Qwen2.5-Math-7B-Instruct under reward-model reranking (∼70%) or reasoning-RL'd models such as o1 (∼74%) and DeepSeek-R1 (∼80%).

MATH-500 evaluation note. For MATH-500, gold answers are LaTeX expressions (fractions, ordered pairs, symbolic). We compare predicted vs gold after normalization: stripping whitespace and `\left`/`\right`, converting `\frac{a}{b}` to `(a)/(b)`, normalizing common operators, and falling back to numeric float

comparison and to SymPy parsing where available. This is a partial implementation of the standard MATH equivalence checker (Lewkowycz et al., 2022); absolute accuracies may be slightly conservative, but all three configurations use the same normalizer so the within-setting comparisons remain valid.

5 Discussion

5.1 The validator's marginal contribution is small, given the python tool

The validator's nominal job is to catch wrong answers via inversion. In practice it does catch some—the val-rate=92.3% / acc | validated=86.69% / acc | unvalidated=78.43% split confirms validation is correlated with correctness. But the ablation shows that the *marginal* contribution of the validator-as-tool to final accuracy, on top of the python tool, is within sampling noise. What the validator does causally is *anchor* the trajectory to a discrete event (`[VALIDATION PASS]`) that the nudge can hook onto. We cannot say from this data whether a different anchor mechanism (e.g., a fixed-turn checkpoint, a model-confidence threshold) would substitute equally well; the prediction would be that any reliable PASS-like event the nudge can fire on should suffice, but this is a hypothesis our experiments do not test.

5.2 Why is the 4B's gain bigger than the 2B's?

A common explanation is that larger models "use tools better." Our data suggest a different story for 4B+Harness's +10.24pp: the 4B teacher's `python`-call rate is much higher than the 2B's (we report val-rate 99.92% and acc | validated 93.93% from a separate run), but the bulk of the marginal gain still goes through the same nudge-driven termination mechanism. The 4B is better at producing a *correct* candidate answer when given more tokens to think in, and so the same "force termination after PASS" lever pays off more. Tool capability is a side-effect, not a driver.

5.3 What we are not claiming

- We are not claiming validators or verifiers are useless in general. Validators that catch *categorically different* failure modes than the forward computation would behave differently in this ablation. The Qwen3.5-2B model rarely produces a candidate that passes inversion but is materially wrong; in domains where it would (e.g., constraint-satisfaction problems), the validator's causal contribution may be much larger.
- We are not claiming this result generalizes to non-arithmetic domains. GSM8K's solutions are short, numeric, and inversion-checkable. On open-ended reasoning, the trade-off changes.

5.4 Implications for scaffolding design

1. **Cheap wins on top of strong base models.** A ∼80-token nudge plus a tool that exposes Python is worth +7.58pp on a saturated 2B. Practitioners shipping math-capable LMs should prioritize this over more elaborate verifier designs.
2. **Negative claim about "smarter tools."** Adding a second tool, or a more sophisticated verifier, may produce no measurable lift over {any Python channel} + {forced termination}—at least on benchmarks of this style. Ablation tables in future scaffolding papers should include the trivial "remove this tool" baseline.
3. **Termination as a first-class design dimension.** Most scaffolding papers treat the loop's exit condition as a footnote ("emit boxed and we parse it"). The ablation here suggests it deserves its own row in the design table.

6 Quantitative Validation Against EFC

The ablation in §4 shows that adding tools does not improve accuracy and adding the post-PASS nudge does. The Effective Feedback Compute framework of Zhang et al. (2026) predicts both findings under a

unified explanation: agent capability tracks the *informative, valid, non-redundant, and retained* fraction of compute, not raw compute itself. We perform a controlled quantitative check on our own traces.

6.1 A binary-Oracle-EFC proxy

The Oracle-EFC per-event formula is $\text{EFC}_t = \kappa I_t V_t R_t M_t$ with $\kappa = 10$ and $I, V, R, M \in [0, 1]$ (Zhang et al., 2026). We compute a binary proxy from trace-observable signals:

- $I_t = 1$ if the event has a tool call or content with a new numeric computation, else 0.
- $V_t = 1$ if a tool returned without `[TOOL ERROR]`, or `[VALIDATION PASS]` fired; $V_t = 0.5$ for content-only turns (no verifiable evidence); $V_t = 0$ on tool errors.
- $R_t = 1$ if this event's tool arguments or content differ from any previous event, else 0.
- $M_t = 1$ if any of this event's emitted numbers appear in the final predicted answer, else 0.

We do not attempt the trained Estimated-EFC of Zhang et al. (2026), whose calibration model is not publicly released. This proxy is a faithful application of the Oracle-EFC product form under conservative binary heuristics; it is not equivalent to the trained estimator and should be read as a lower-bound sketch.

6.2 Per-sample correlation across configurations

We re-run four GSM8K configurations on Qwen3.5-2B+OPD+GRPO with $n = 500$ each, saving full traces (total 1950 traces after dropping any with parse errors). The configurations span the central ablation: full Harness (acc=85.20, mean EFC= 24.14), −python only (87.00, 17.82), −nudge (83.80, 25.16), −no-tools (84.60, 5.17).

Per-sample point-biserial correlation (Table 6). Three trace-level scalars are tested against the binary success indicator:

Table 6: Per-sample correlation between trace-level scalar predictors and binary correctness, pooled across four GSM8K configurations ($n = 1950$).

Predictor	r_{pb}
Binary-Oracle-EFC proxy	+0.016
Tool calls per trace	+0.022
Raw token count	−0.215

Raw token count is negatively correlated with correctness, while EFC and tool calls hover near zero at the pooled level. This is the central qualitative claim of (Zhang et al., 2026) replicated in our setting: more raw compute does not predict more capability, and in fact predicts *less*—traces that grow longer are disproportionately the ones that fail.

6.3 Accuracy by EFC quartile

A point-biserial correlation can be near zero while a monotone non-linear effect is present. We bucket the 1950 pooled traces into EFC quartiles and report per-bucket accuracy.

Q4 accuracy is 6.98pp above Q1 and 9.86pp above Q2. Beyond Q2 the monotone-increasing pattern is clean: more retained, validated, non-redundant feedback per trace predicts higher per-trace accuracy. The Q1 anomaly (high accuracy at near-zero EFC) is contributed almost entirely by no-tools traces, which produce very few feedback events but solve the task via direct CoT—a regime where the EFC scalar is not informative because the harness is open-loop.

Table 7: Pooled accuracy by EFC quartile (n = 1950, quartiles by per-trace EFC). Q1 is dominated by no-tools traces (very low event counts); Q4 is dominated by full-Harness and −nudge traces with many retention-positive events.

Quartile	mean EFC	n	Accuracy
Q1 (lowest)	5.00	487	85.63%
Q2	15.57	487	82.75%
Q3	22.01	487	88.30%
Q4 (highest)	31.29	487	**92.61%**

6.4 Interpretation

Three observations cohere:

1. At the config-mean level, raw tokens *negatively* predict accuracy (r_{pb}=−0.215). Adding more tool calls and more turns systematically associates with failure, not success.
2. Among closed-loop traces (Q2–Q4), the EFC proxy predicts a +9.86pp accuracy gain from low to high.
3. Open-loop traces (Q1, dominated by no-tools) sit at the same accuracy as closed-loop high-EFC traces (85.63% vs 92.61% via a different route)—confirming that what matters is feedback *utility*, not feedback *quantity*.

This is the empirical content the EFC framework predicts (Zhang et al., 2026): at fixed model and task, capability tracks the effective rather than the raw fraction of compute. Our ablation in §4 can now be read in EFC vocabulary: the post-PASS nudge is a *retention/use operator* that converts the latent `[VALIDATION PASS]` event into a binding termination decision, raising the fraction of the trajectory that is retained and acted upon without changing the raw compute budget.

7 Related Work

Tool-augmented reasoning for math. Program-of-Thought (Chen et al., 2022) introduced executing Python as the *primary* reasoning path. ToRA (Gou et al., 2023) alternates between reasoning and Python and trains on tool-use trajectories. Self-verification approaches (Weng et al., 2022) check candidate answers via an auxiliary model or reversed problem. Our harness combines these ideas, then asks which ingredient matters.

On-policy distillation. Agarwal et al.'s OPD (Agarwal et al., 2023) is the immediate predecessor of our backbone. Liu et al.'s Dr.GRPO (Liu et al., 2025) removes length and within-group variance normalization in GRPO; DAPO (Yu et al., 2025) introduces asymmetric clipping. Our backbone combines all three.

Negative results for "agent" components. Concurrent work has noted that complex agent loops sometimes underperform CoT and that verifier choice is less important than expected in RL-from-feedback. Our finding sharpens this: in a specific clean setup, the verifier's causal contribution is near zero, and the actual lever is a single control-flow token.

Effective Feedback Compute. Concurrent work (Zhang et al., 2026) formalizes a complementary scaling law: agent capability tracks not raw compute (tokens, tool calls, wall time) but the fraction of that compute that produces feedback the agent actually retains and uses to change its next decision. Their Oracle-EFC, normalized by per-task feedback demand, explains R^2=0.99 of the variance in agent performance across synthetic and executable tasks, versus R^2=0.33 for raw token counts and R^2=0.42 for raw tool calls. Our paper offers a mechanism-level instance of this finding within a single fixed harness: the post-PASS nudge

does not add tools, tokens, or model capacity, but it converts a previously-ignored `[VALIDATION PASS]` signal into a binding termination decision. In EFC vocabulary, the nudge is a *retention/use* operator: it raises the fraction of the trajectory that is informative, non-redundant, and acted upon, without changing the raw compute budget. The +4.25pp −nudge gap and the near-zero individual-tool gaps are exactly what an EFC-style account predicts—adding raw compute in the form of an extra tool does not move the needle; adding a control-flow primitive that increases retention does.

8 Limitations and Outlook

- **Limited dataset coverage.** GSM8K and SVAMP both involve short numeric solutions with easy inversion. Harder benchmarks (MATH-500, AIME, OlympiadBench) might show the validator's marginal contribution recovering, especially on problems where the forward CoT is unreliable but the inversion check is not. Whether the "tools don't help, often hurt" pattern holds on those benchmarks is an open empirical question.
- **Two model families only.** We test Qwen3.5 (trained student) and Llama-3.2-3B-Instruct (off-the-shelf). The cross-family extrapolation is one data point in each direction; Mistral, Phi, and Gemma would each be informative.
- **Tool design space narrow.** We ablate tool *removal*, not tool *variation*. A richer ablation (different validator designs, content-free vs. content-bearing nudges, longer `max_turns`) would more sharply isolate the nudge's mechanism.
- **Binary EFC proxy, not trained estimator.** §6 uses a coarse binary heuristic to compute the Oracle-EFC factors. The trained Estimated-EFC of (Zhang et al., 2026) would likely produce stronger correlations; our proxy should be read as a faithful but lower-bound application of the framework, not as a replication of their results.
- **Reward in-distribution.** The 2B backbone was trained on GSM8K specifically. The OOD gain on SVAMP (+3.5pp from removing tools) is suggestive but a backbone trained on MATH and evaluated on GSM8K (or vice versa) would isolate the "what is the model already saturated on?" axis more sharply.

9 Conclusion

Inference-time scaffolding lifts a 2B GSM8K model from 78.47% to 86.05%—a gain larger than the gap to the 4B teacher at base CoT. We then add the no-tool, nudge-active cell to the ablation, replicate the comparison on SVAMP OOD, and replicate it again on a cross-family Llama-3.2-3B-Instruct. Three findings cohere across all settings:

1. **The +7.58pp lift on Qwen GSM8K is fully recovered without any tool execution at all** (86.20% no-tools vs 86.05% full Harness).
2. **On the harder transfer settings, removing the tools *improves* accuracy substantially**: +3.50pp on SVAMP, +3.40pp on MATH-500, +15.62pp on Llama.
3. **The post-PASS nudge is a control-flow patch whose role is to neutralize a tool-induced failure mode**, not to add capability. It matters only on the configurations where the model otherwise spirals after `[VALIDATION PASS]`.

A trace-level binary-Oracle-EFC proxy validates the broader scaling law of Zhang et al. (2026) in our setup: raw tokens negatively correlate with success (r_{pb}=−0.215), while accuracy by EFC quartile rises monotonically from 85.6% (Q1) to 92.6% (Q4). The field has been over-crediting tool design and under-crediting termination control; future scaffolding work should ablate the tools *out*, not just *in*, and should report the trace-level effective-feedback fraction alongside raw compute.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. *arXiv:2306.13649*, 2023.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv:2211.12588*, 2022.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. *arXiv:2309.17452*, 2023. ICLR 2024.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. *arXiv:2206.14858*, 2022.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. *arXiv:2503.20783*, 2025.

Yixuan Weng, Minjun Zhu, Fei Xia, Bin Li, Shizhu He, Shengping Liu, Bin Sun, Kang Liu, and Jun Zhao. Large language models are better reasoners with self-verification. *arXiv:2212.09561*, 2022. EMNLP 2023 Findings.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv:2503.14476*, 2025.

Xuanliang Zhang, Dingzirui Wang, Keyan Xu, Qingfu Zhu, and Wanxiang Che. Scaling laws for agent harnesses via effective feedback compute. *arXiv:2605.29682*, 2026.

A Per-ablation auxiliary metrics

Config	n	Acc	Val rate	Acc | val	Acc | unval	py%	val%
Harness (full)	1319	86.05	92.3	86.69	78.43	68.4	96.0
− python	1319	86.58	94.0	88.87	50.63	0.0	95.3
− validate	1319	86.20	0.0	—	86.20	10.9	0.0
− nudge	1319	81.80	89.5	86.96	37.68	63.2	91.0

Notable: in −python, the *unvalidated* acc drops to 50.63% (the model has no fallback Python and gets unvalidated answers wrong). In −nudge, *unvalidated* acc drops to 37.68%—these are the trajectories that exhausted `max_turns` without ever boxing.

B Worked failure mode without nudge

A representative −nudge trajectory after `[VALIDATION PASS]`:

```
[turn 5] validate -> [VALIDATION PASS] (claimed=18, verified=18)
[turn 6] assistant: <tool_call><function=python><parameter=code>
            # Let me double-check by computing it a third way
            ...
```

```
[turn 7] assistant: <tool_call><function=validate>... (re-validates same answer)
[turn 8] assistant: <tool_call><function=python>...
...
[turn 10] coda: "Stop calling tools, output boxed now (validated)."
[turn 10] assistant: "\boxed{16}"   <- changed answer in the coda call
```

The model after PASS often re-checks until it accidentally talks itself out of the correct answer in the forced coda turn. The nudge cuts the trajectory off at turn 6 with `[VALIDATION PASS received] STOP calling tools immediately. Output \boxed{18}.`, preserving the validated answer.